

Coq Proof Assistant	LeanProver	Meaning...
apply	apply	Apply theorem/hypothesis to hypothesis/goal state
intros	intros	Adds argument of goal-state-theorem to context
assumption	assumption	Similar to exact; solve ‘by assumption’ from context
unfold	unfold	If H: $x=y$, swaps y for x in a goal state (or elsewhere)
contradiction	contradiction	If H: $x=y$, swaps x for y in a goal state (or elsewhere)
exact	exact	If a proof of the desired goal is in context, exact provides it
try	try	Applies a tactic to the goal only if that tactic solves the goal. Useful in complex composite tactics
Print	#print	Gives the definition of the thing proceeding it
Locate	??	Gives the location of the definition of the thing proceeding it
Search	??	Gives all other instances of the thing proceeding it
inversion	Closest analogy is cases	Essentially asks the proof assistant ‘what had to be true in order for this proposition to come about’ and case breaks on all such possibilities
assert	??	Creates a new hypothesis/subproof of your own requirements and creates a new goal that you prove it
repeat	repeat	Transforms a tactic into a version of the tactic that repeats multiple times (forms complex tactics with ; composition)
eapply	apply	Similar to apply but slower, allowing apply to search for candidates to fill existentials
eauto	eauto	Normal auto can only use tactics: intros, apply, assumption, reflexive. Eauto can use eapply.
left right	left right	Takes goal state P OR Q and tells PA with left we’ll try to prove P, or with right that we’ll try to prove Q.

Coq Proof Assistant	LeanProver	Meaning...
Theorem	theorem	Proposition function
admit	sorry	Tells PA you’re coming back to this later
reflexivity	rfl	Completes goal from $x=x$ state
rewrite H	rw [H]	If H: $x=y$, swaps y for x in a goal state (or elsewhere)
rewrite <- H	rw [<- H]	If H: $x=y$, swaps x for y in a goal state (or elsewhere)
simpl, cbn, auto	simp, dsimp	Simpl applies any function reductions that are available. Auto attempts to complete the proof using simple tactics.
destruct, case, elim	cases	Case-break on different inductive constructors and create new goals for each case
discriminate	contradiction	For trying to show the goal by contradiction
specialize	specialize	Add a theorem to context which has been specified to contextual arguments
subst	subst_vars	Eliminate unnecessary equalities by replacing/renaming as many symbols as necessary
;	<;>	Compose two tactics into one
- + * 1,3,4: all:	\.	Ways to focus goals or target tactics to goals
as	??	Allows tactics that transform hypotheses to form a new hypothesis instead of consuming one from context
in	at	Targets some tactics to hypotheses in context instead of goal
generalize dependent	revert	Return an item from context back to the goal state, often in order to control what you induction over
split	apply And.intro	Splits proofs of P AND Q into two goals
symmetry	apply Eq.symm	Swaps the two sides of an equality either in the goal or in a hypothesis
f_equal	apply congrArg	Turns a proof of $x=y$ and a function f into a proof of $f(x)=f(y)$